Diffusion models

План

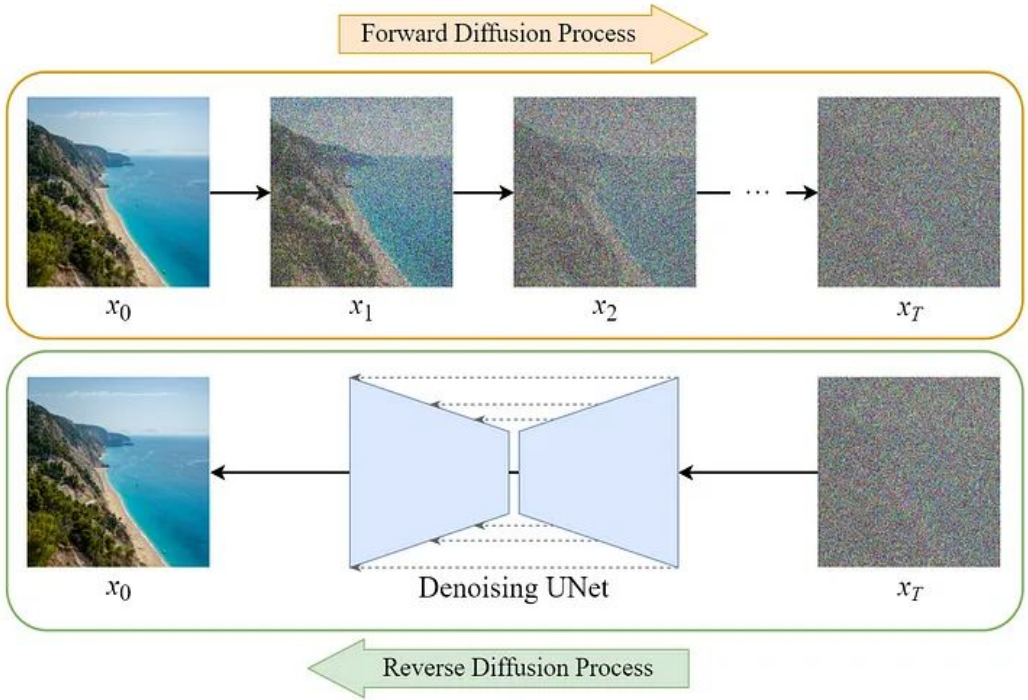
- DDPM
- ускоряем процесс сэмплирования DDIM
- улучшаем качество генерации Classifier free guidance
- Уменьшаем размер модели LDM
- Stable diffusion (Condition, txt2img, img2img, inpaint)
- Diffusion Transformers
- Эволюция SD1.4 -> SD1.5 -> SDXL -> SD3 -> FLUX

links

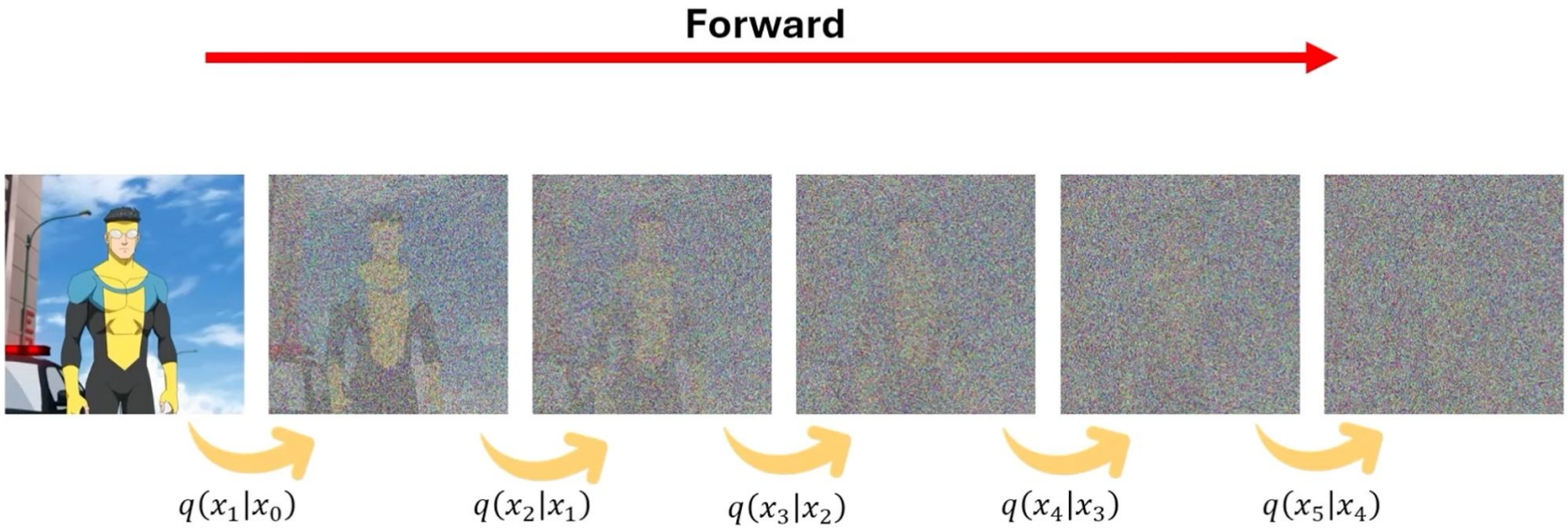
- https://jalammar.github.io/illustrated-stable-diffusion/
- https://www.youtube.com/watch?v=ltLNYA3lWAQ
- https://github.com/castorini/daam
- https://www.youtube.com/watch?v=J2WtkA1Xfew
- https://vaclavkosar.com/ml/cross-attention-in-transformer-architecture
- https://arxiv.org/abs/2112.10752
- https://sander.ai/2022/05/26/guidance.html
- https://stable-diffusion-art.com/how-stable-diffusion-work/#How_training_is_done
- https://theaisummer.com/diffusion-models/
- https://education.yandex.ru/handbook/ml/article/diffuzionnye-modeli
- https://arxiv.org/pdf/2403.18103.pdf
- https://strikingloo.github.io/wiki/ddim
- https://nn.labml.ai/diffusion/stable_diffusion/index.html
- https://colab.research.google.com/github/huggingface/notebooks/blob/main/diffusers/diffusers_intro.ipynb

DDPM (Denoising Diffusion Probabilistic Models)

1. Forward Diffusion Process → добавить шум к изображению.
2. Reverse Diffusion Process →убрать шум из изображения .

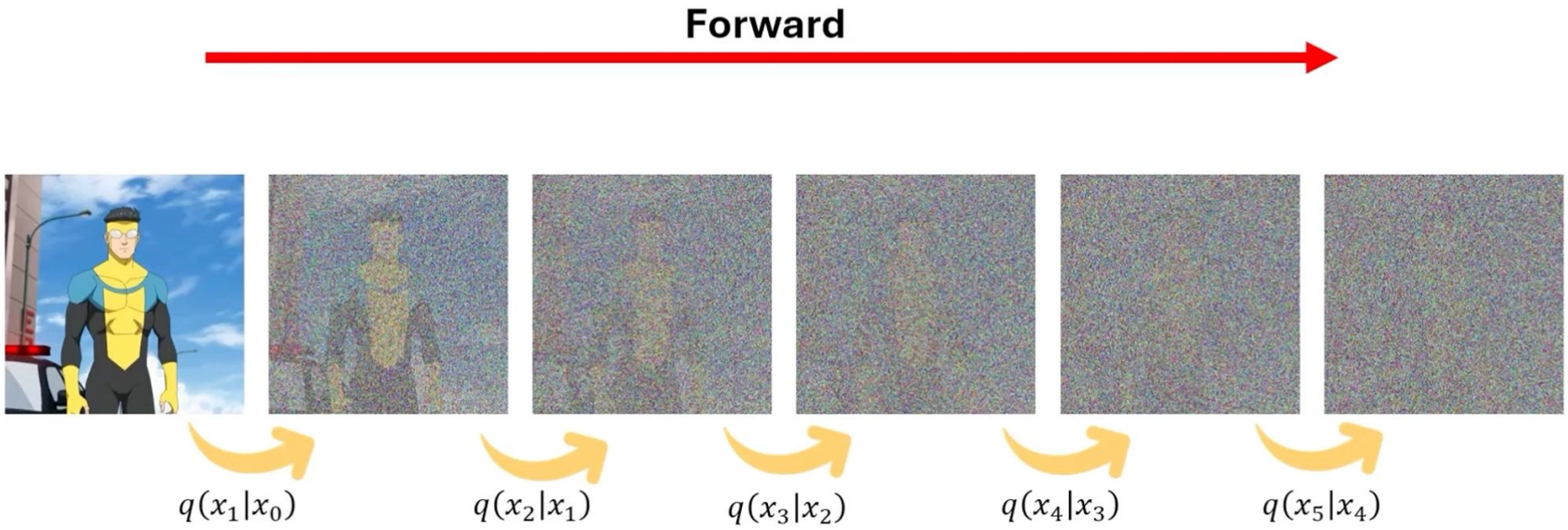


DDPM (Denoising Diffusion Probabilistic Models)



$$q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0) := \prod_{t=1}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$$

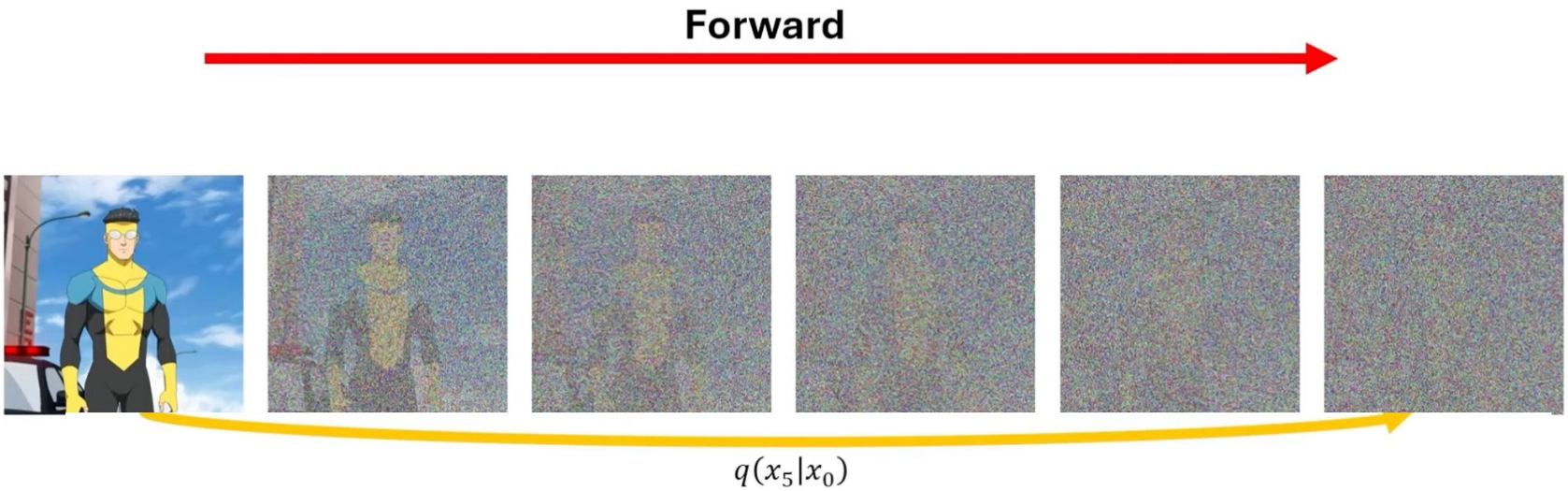
DDPM (Denoising Diffusion Probabilistic Models)



$$q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0) := \prod_{t=1}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$$

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) := \mathcal{N}\left(\sqrt{\frac{\alpha_t}{\alpha_{t-1}}}\boldsymbol{x}_{t-1}, \left(1 - \frac{\alpha_t}{\alpha_{t-1}}\right)\boldsymbol{I}\right)$$

DDPM (Denoising Diffusion Probabilistic Models)

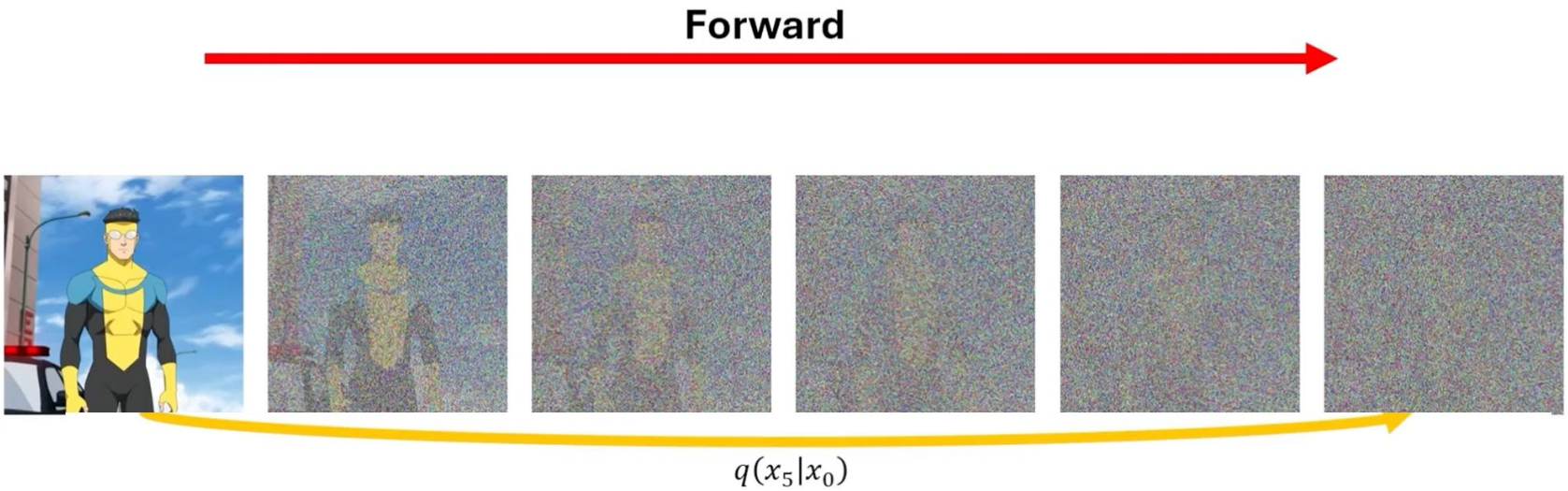


$$q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0) := \prod_{t=1}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$$

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) := \mathcal{N}\left(\sqrt{\frac{\alpha_t}{\alpha_{t-1}}}\boldsymbol{x}_{t-1}, \left(1 - \frac{\alpha_t}{\alpha_{t-1}}\right)\boldsymbol{I}\right)$$

$$q(\boldsymbol{x}_t|\boldsymbol{x}_0) := \mathcal{N}(\boldsymbol{x}_t; \sqrt{\alpha_t}\boldsymbol{x}_0, (1-\alpha_t)\boldsymbol{I})$$

DDPM (Denoising Diffusion Probabilistic Models)



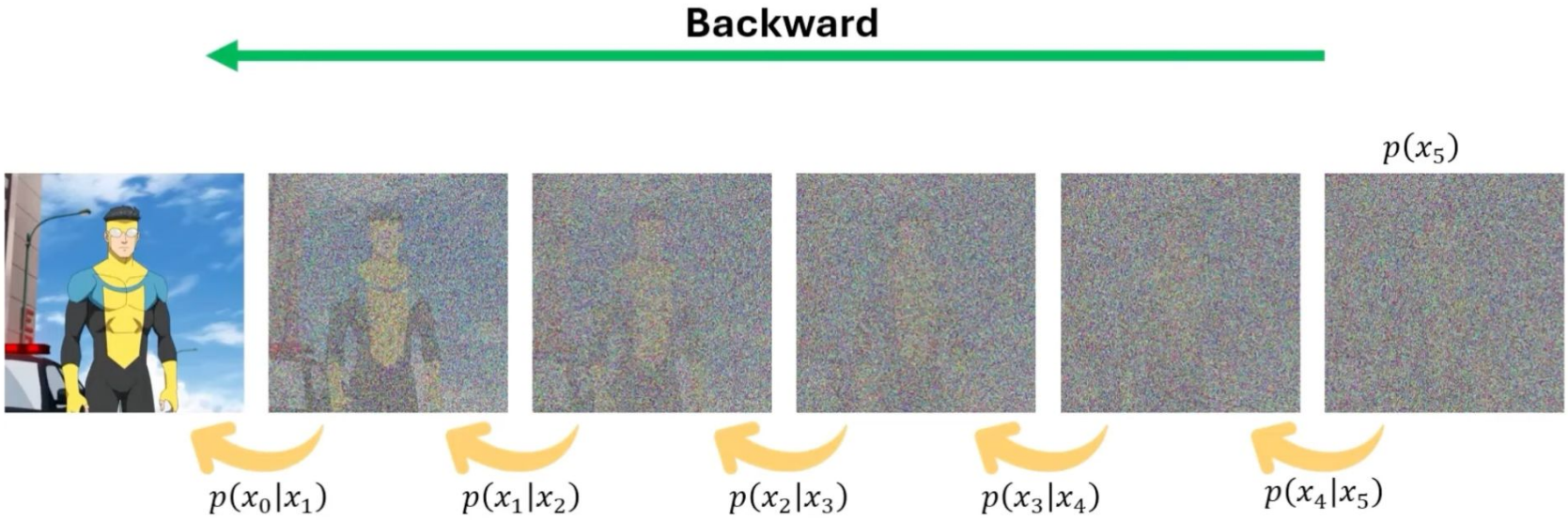
$$q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0) := \prod_{t=1}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$$

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) := \mathcal{N}\left(\sqrt{\frac{\alpha_t}{\alpha_{t-1}}}\boldsymbol{x}_{t-1}, \left(1 - \frac{\alpha_t}{\alpha_{t-1}}\right)\boldsymbol{I}\right)$$

$$q(\boldsymbol{x}_t|\boldsymbol{x}_0) := \mathcal{N}(\boldsymbol{x}_t; \sqrt{\alpha_t}\boldsymbol{x}_0, (1-\alpha_t)\boldsymbol{I})$$

$$\boldsymbol{x}_t = \sqrt{\alpha_t}\boldsymbol{x}_0 + \sqrt{1-\alpha_t}\epsilon, \quad \text{where} \quad \epsilon \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$$

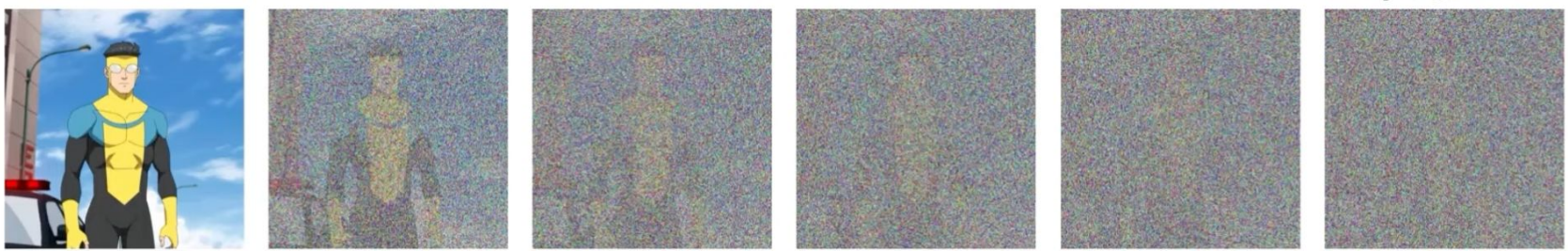
DDPM (Denoising Diffusion Probabilistic Models)



$$p_\theta(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

DDPM (Denoising Diffusion Probabilistic Models)



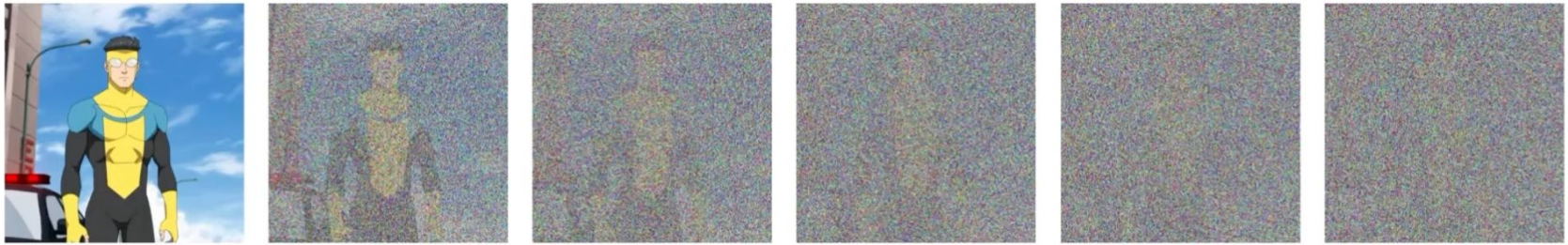
Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on

$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$

6: **until** converged

DDPM (Denoising Diffusion Probabilistic Models)



Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on

$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$

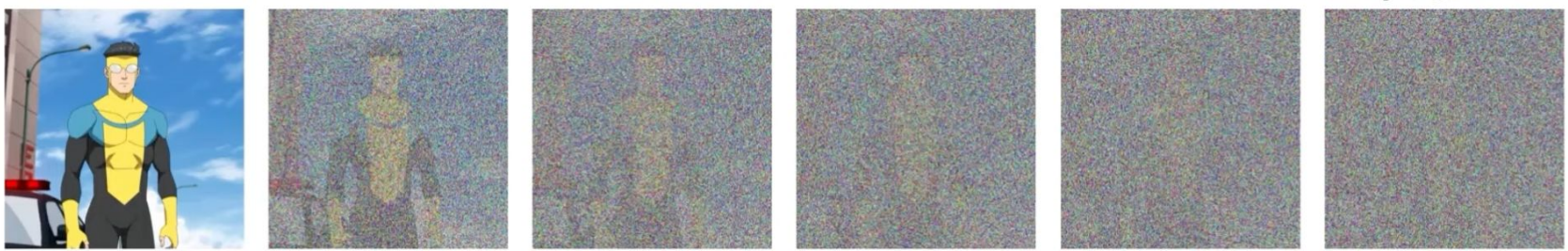
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

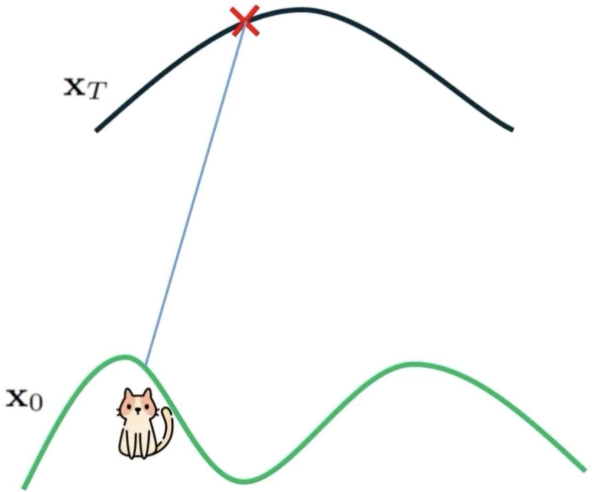
$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) := \mathcal{N}\left(\sqrt{\frac{\alpha_t}{\alpha_{t-1}}}\boldsymbol{x}_{t-1}, \left(1 - \frac{\alpha_t}{\alpha_{t-1}}\right)\boldsymbol{I}\right)$$

DDPM (Denoising Diffusion Probabilistic Models)

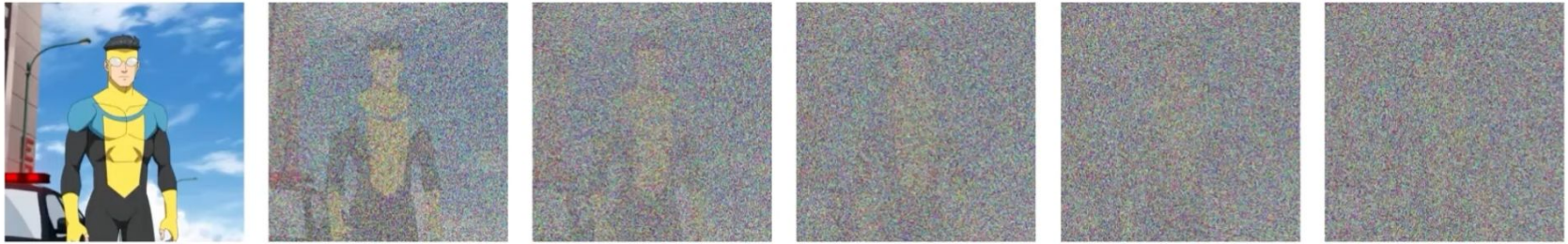


Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

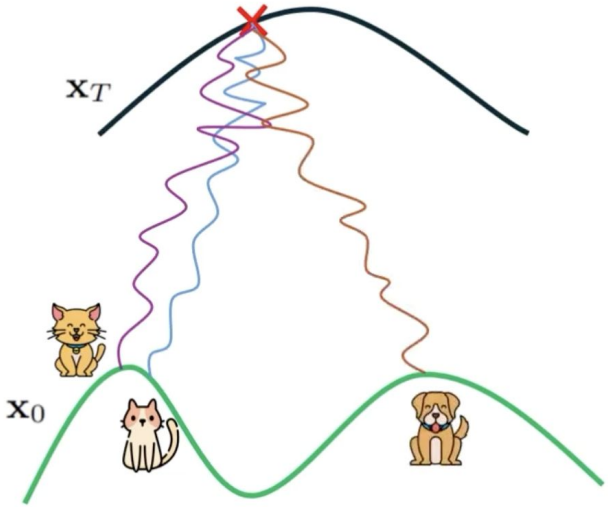


DDPM (Denoising Diffusion Probabilistic Models)



Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$



DDPM (Denoising Diffusion Probabilistic Models)

❌ Проблемы DDPM:

- Очень медленно, нужно пройти в T часто 1000 шагов
- Каждый шаг — добавляем новый случайный шум, что делает процесс непредсказуемым
- Нет контроля над траекторией пока не пройдем весь путь до x0

DDIM (Denoising Diffusion Implicit Models)

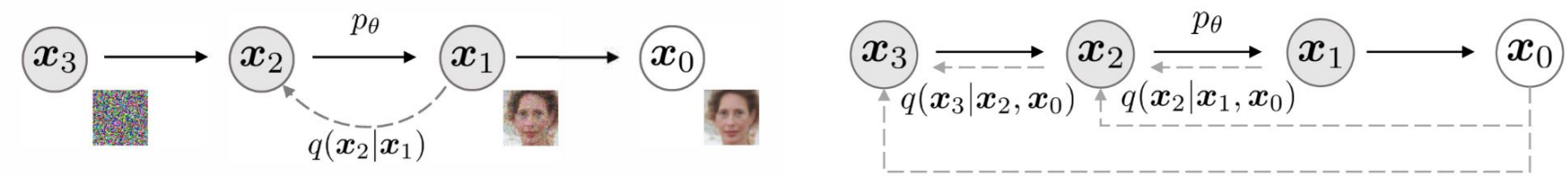


Figure 1: Graphical models for diffusion (left) and non-Markovian (right) inference models.

В Марковской цепи это последовательность случайных состояний, в которой **будущее зависит только от настоящего**, а не от прошлого

$$\boldsymbol{x}_{t-1} = \sqrt{\alpha_{t-1}} \underbrace{\left(\frac{\boldsymbol{x}_t - \sqrt{1-\alpha_t}\epsilon_\theta^{(t)}(\boldsymbol{x}_t)}{\sqrt{\alpha_t}} \right)}_{\text{“ predicted } \boldsymbol{x}_0 \text{”}} + \underbrace{\sqrt{1-\alpha_{t-1}-\sigma_t^2} \cdot \epsilon_\theta^{(t)}(\boldsymbol{x}_t)}_{\text{“direction pointing to } \boldsymbol{x}_t \text{”}} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}}$$

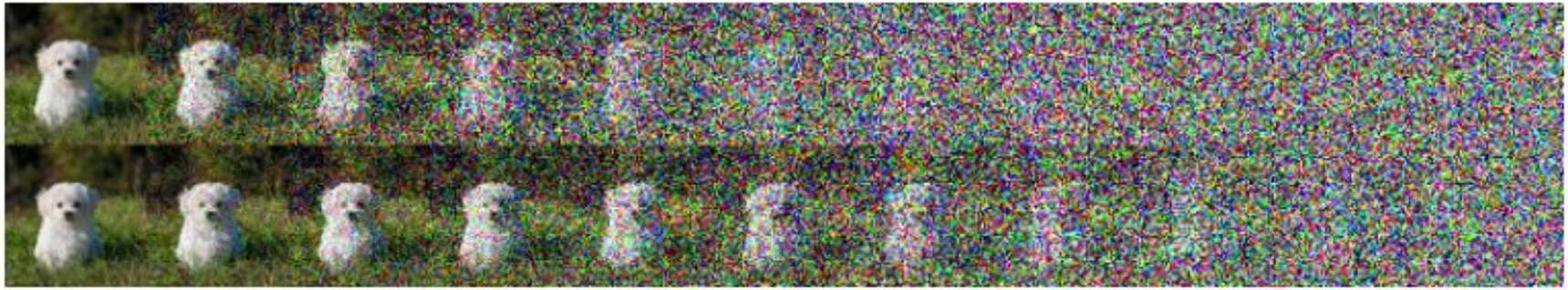
DDIM (Denoising Diffusion Implicit Models)

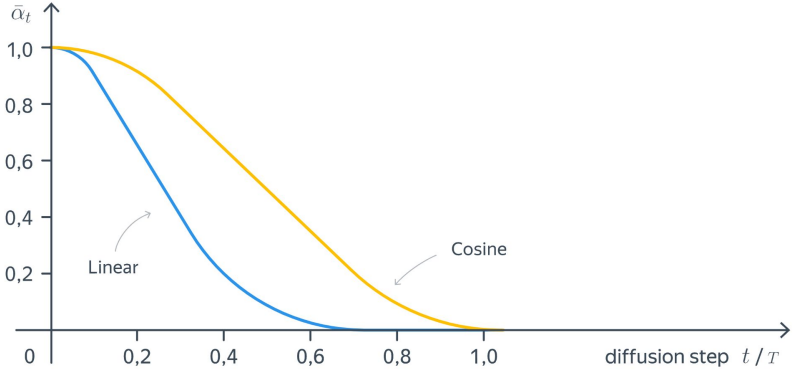
$$\boldsymbol{x}_{t-1} = \sqrt{\alpha_{t-1}} \underbrace{\left(\frac{\boldsymbol{x}_t - \sqrt{1-\alpha_t}\epsilon_\theta^{(t)}(\boldsymbol{x}_t)}{\sqrt{\alpha_t}} \right)}_{\text{“ predicted } \boldsymbol{x}_0 \text{”}} + \underbrace{\sqrt{1-\alpha_{t-1}-\sigma_t^2} \cdot \epsilon_\theta^{(t)}(\boldsymbol{x}_t)}_{\text{“direction pointing to } \boldsymbol{x}_t \text{”}} + \underbrace{\sigma_t \epsilon_t}_{\text{random noise}}$$

Step 0 | Step 181 | Step 381 | Step 581 | Step 781 | Step 999

https://www.youtube.com/watch?v=r4V0vLhYZIQ&t=23s https://arxiv.org/pdf/2010.02502

Improved Denoising Diffusion Probabilistic Models





https://arxiv.org/abs/2102.09672

Classifier-free guidance

Все, что нам нужно, чтобы превратить модель безусловной диффузии в условную, — это классификатор!

В отличие от вероятностных моделей на прямую аппроксимирующие p(x), диффузионная модель предсказывает score function

$$\nabla_x \log p(x \mid y).$$

Classifier guidance

$$\nabla_x \log p_\gamma(x \mid y) = \nabla_x \log p(x) + \gamma \nabla_x \log p(y \mid x).$$

Classifier-free guidance

$$\nabla_x \log p_\gamma(x \mid y) = \nabla_x \log p(x) + \gamma \left(\nabla_x \log p(x \mid y) - \nabla_x \log p(x) \right),$$

https://sander.ai/2022/05/26/guidance.html https://arxiv.org/pdf/2207.12598

Classifier-free guidance

Samples from an unconditional diffusion model with classifier guidance, for guidance scales 1.0 (left) and 10.0 (right), taken from Dhariwal & Nichol (2021).

Classifier guidance

```
classifier_model = ...  # Load a pre-trained image classification model
y = 1  # We want to generate an image of class 1, let's assume class 1 correspon
guidance_scale = 7.5  # Controls the strength of the class guidance, the higher
input = get_noise(...)  # Randomly draw noise with the same shape as the output

for t in tqdm(scheduler.timesteps):

    # Use neural networks for inference, to predict noise (score, the first term
    with torch.no_grad():
        noise_pred = model(input, t).sample

    # Classifier guidance step
    class_guidance = classifier_model.get_class_guidance(input, y) # Compute gra
    noise_pred+= class_guidance * guidance_scale  # Apply the gradient

    # Calculate x_{t-1} using the updated noise
    input = scheduler.step(noise_pred, t, input).prev_sample
```

https://medium.com/@baicenxiao/understand-classifier-guidance-and-classifier-free-guidance-in-diffusion-model-via-python-e92c0c46ec18

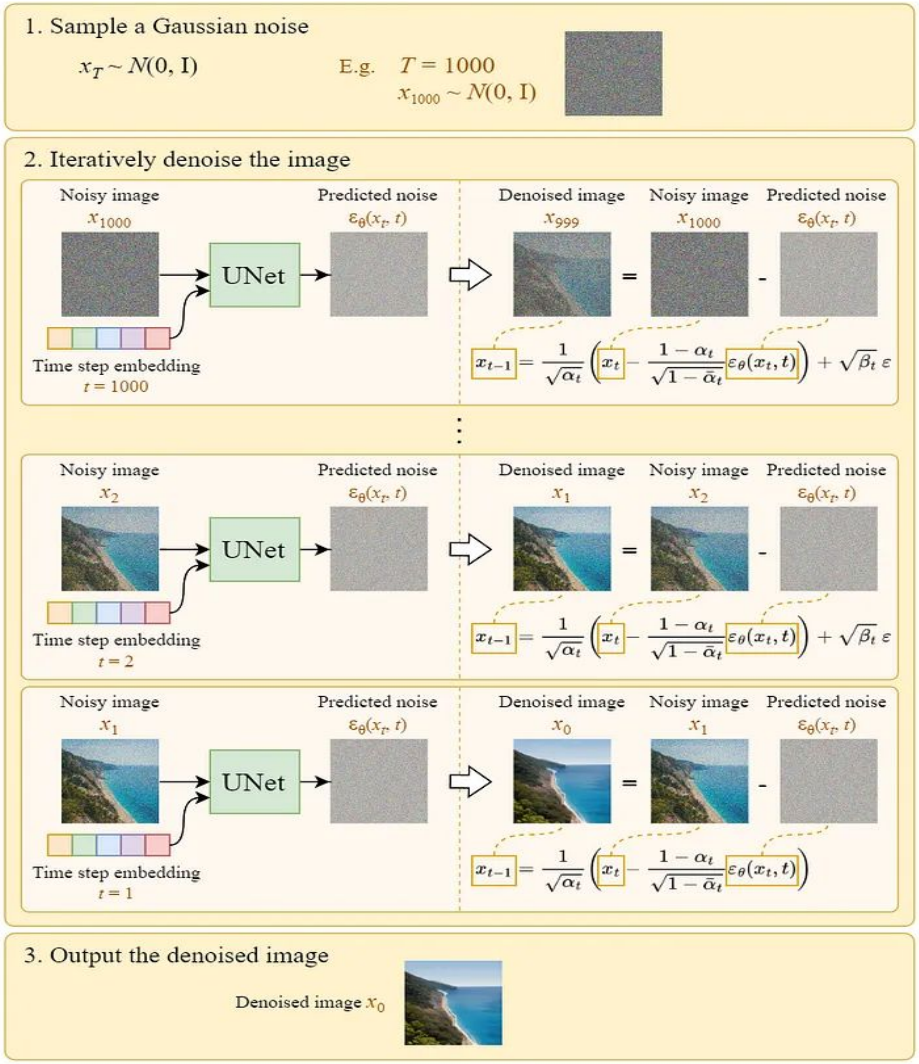
Training step

Генерируем гауссовский шум

Итеративно избавляемся от шума пока не получим исходное изображение

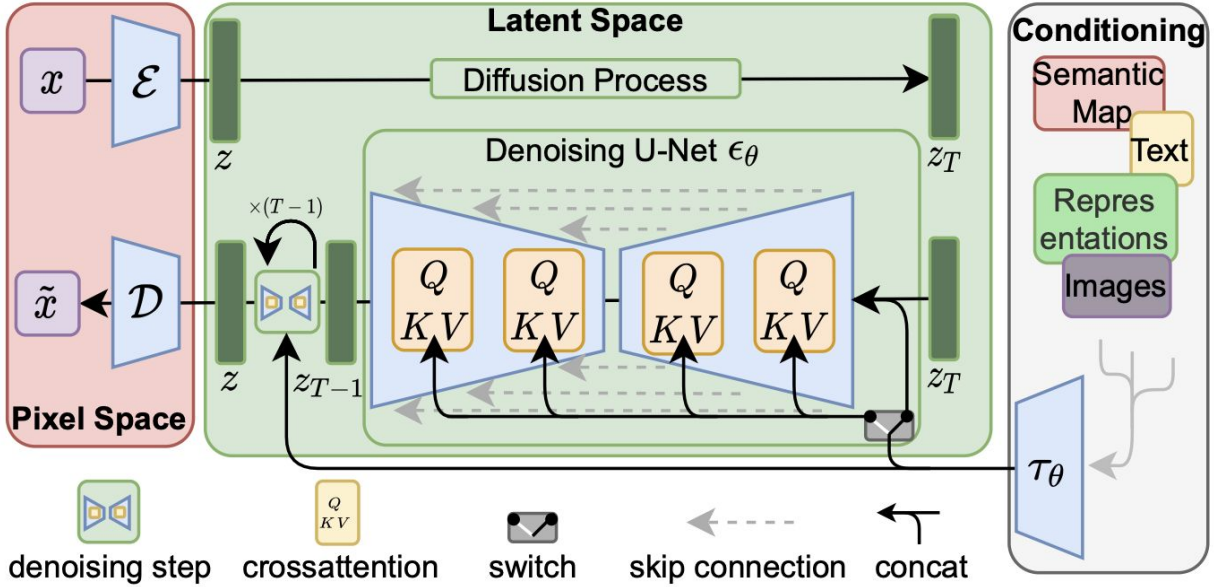
Недостатки:

Работа с целым изображением вычислительно очень медленная и дорогостоящая операция



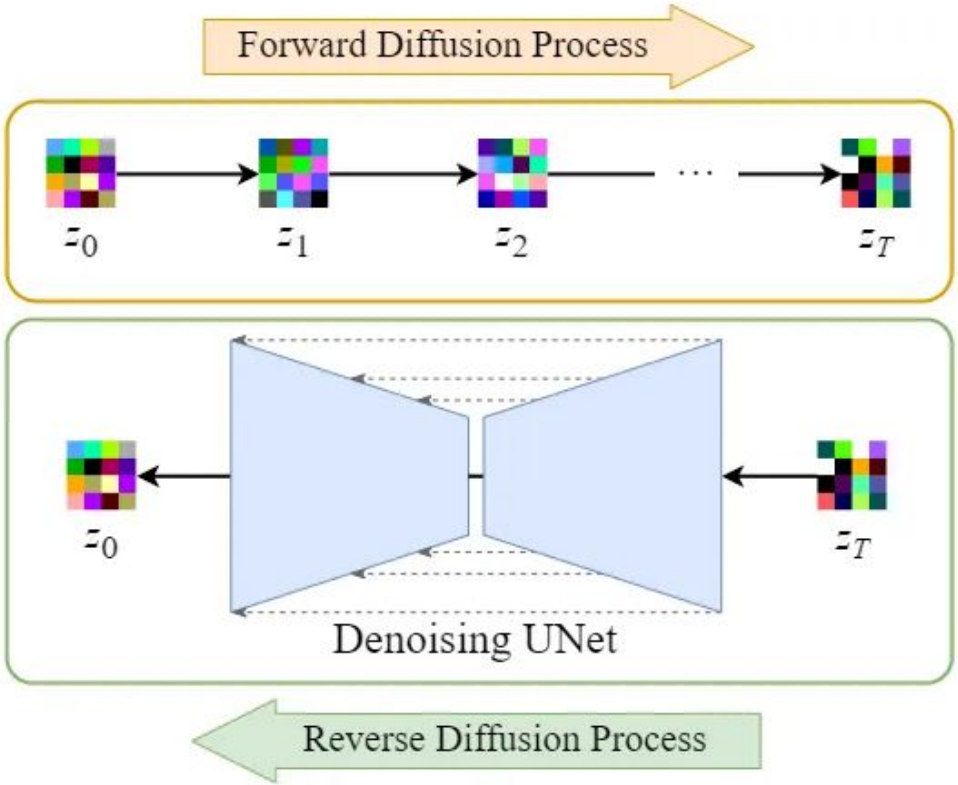
High-Resolution Image Synthesis with Latent Diffusion Models

- Encoder/Decoder
- Latent Space
- UNet Scheduler
- Conditioning



Latent Diffusion

1. Forward Diffusion Process → добавить шум к латенту.
2. Reverse Diffusion Process →убрать шум из латента.

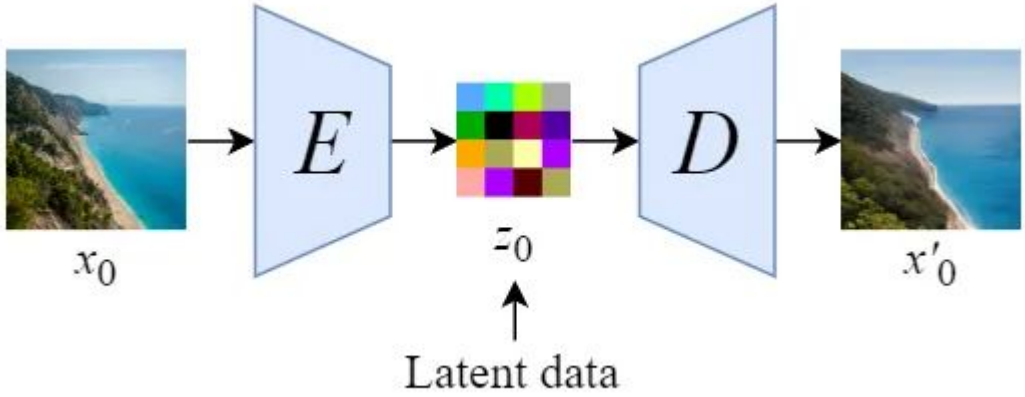


Latent Space

процесс диффузии теперь выполняется в латентном пространстве

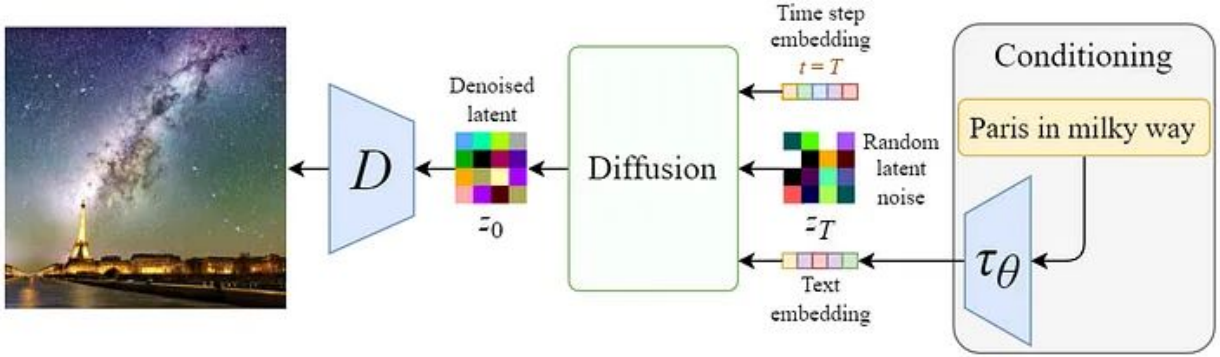
- encoder ***E*** кодирует изображение
- decoder **D** декодирует

изображение(3, 512, 512) -> латент(4,64,64)



Conditioning

Добавление в процесс диффузии (условия) в данном случае текста с использованием механизма **cross-attention** Для генерации новых изображений по текстовому описанию

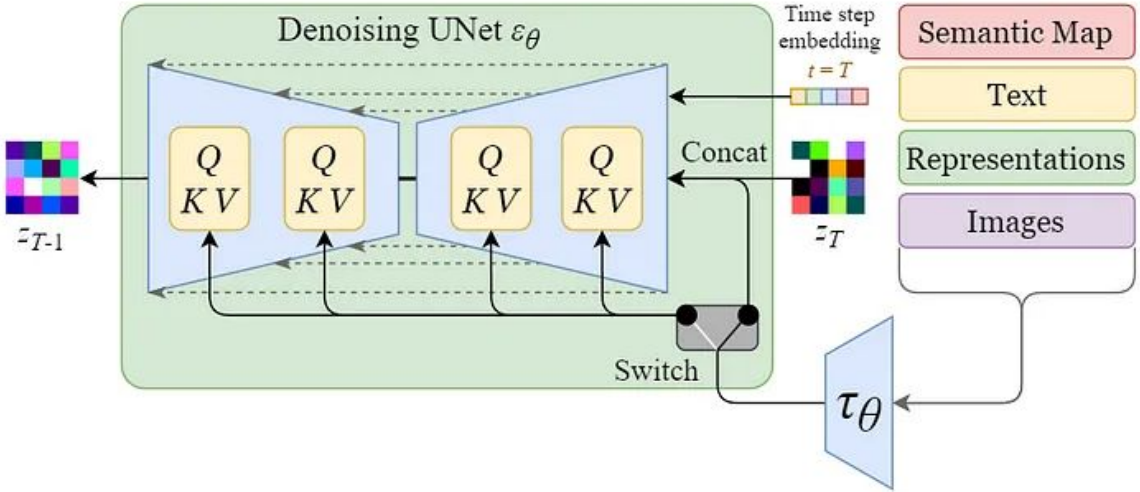


Conditioning

switch нужен для использование разного типа “условий” на вход

для текста используется языковая модель для получения вектора (BERT, CLIP) и с помощью Attention мапится в UNet

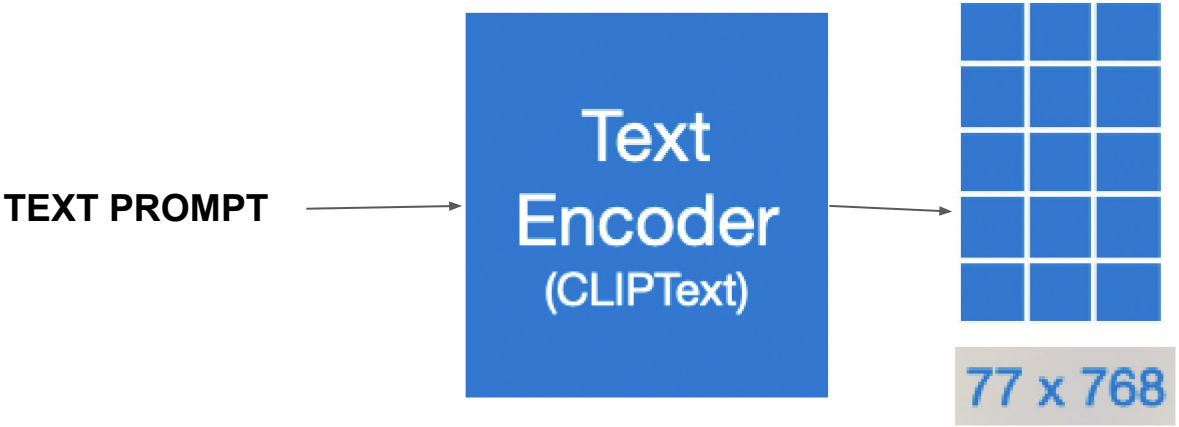
Для другого рода входов (семантические маски, изображения) может использоваться конкатинация



ClipText

Input: text.

Output: 77 token embeddings vectors, each in 768 dimensions.



Loss function

$$x_t = \sqrt{\bar{a}_t}\, x_0 + \sqrt{1 - \bar{a}_t}\, \varepsilon$$

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \boldsymbol{\varepsilon}} \left[\| \boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(x_t, t) \|^2 \right]$$

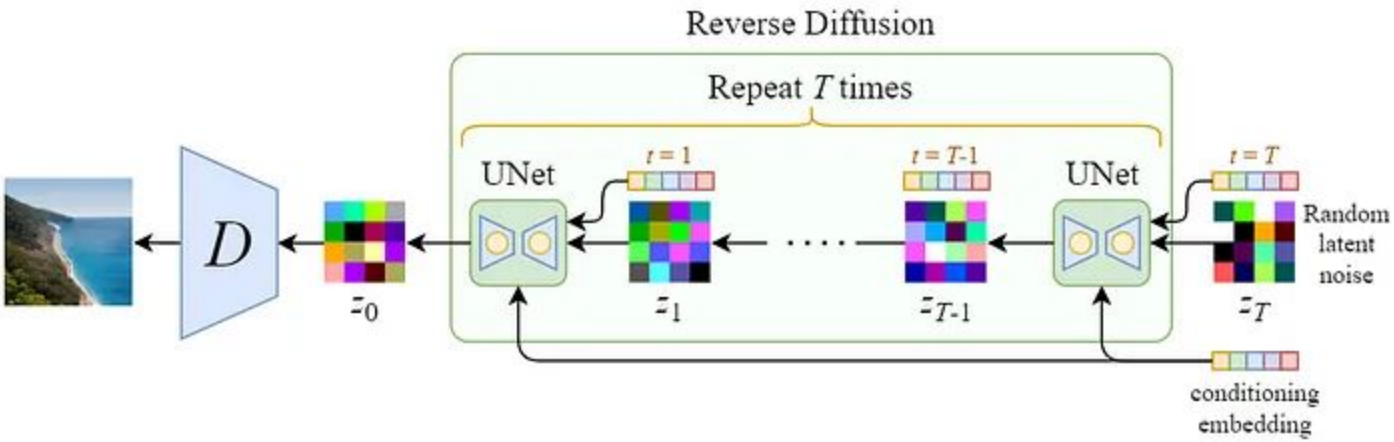
$$z_0 = E(x_0)$$

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon$$

$$L_{\text{LDM}} = \mathbb{E}_{t, z_0, \boldsymbol{\varepsilon}, y} \left[\| \boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_\theta(z_t, t, \tau_\theta(y)) \|^2 \right]$$

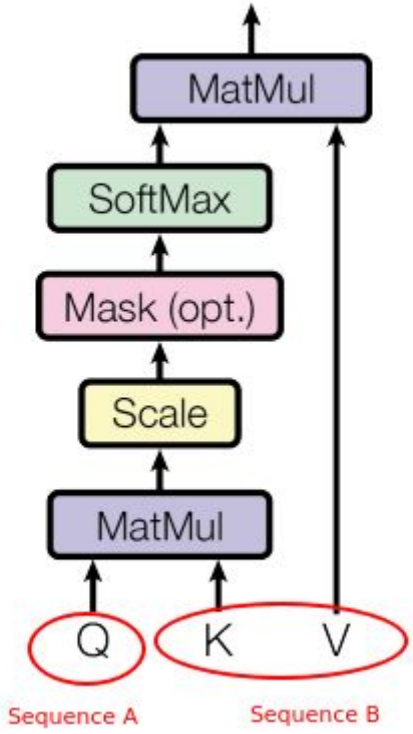
Conditioning

Sampling

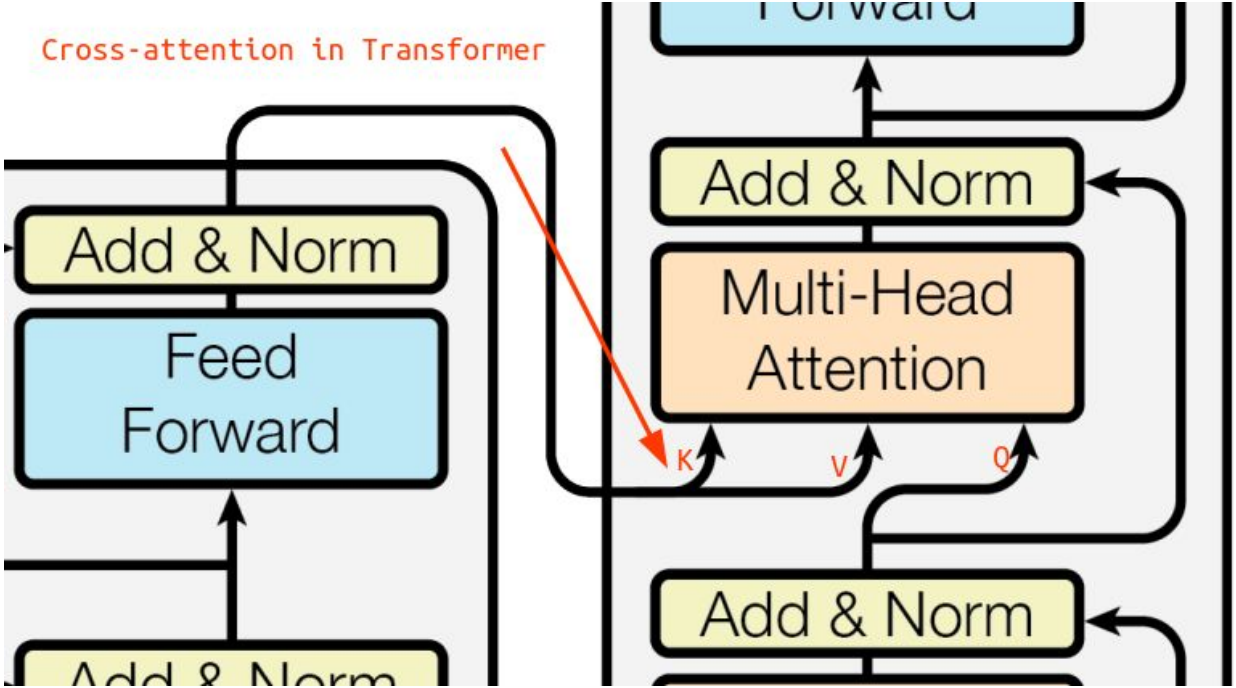

Reverse Diffusion
Repeat T times
UNet
t = 1
t = T-1
UNet
t = T
Random
latent
noise
D
z0
z1
zT-1
zT
conditioning
embedding

Cross attention

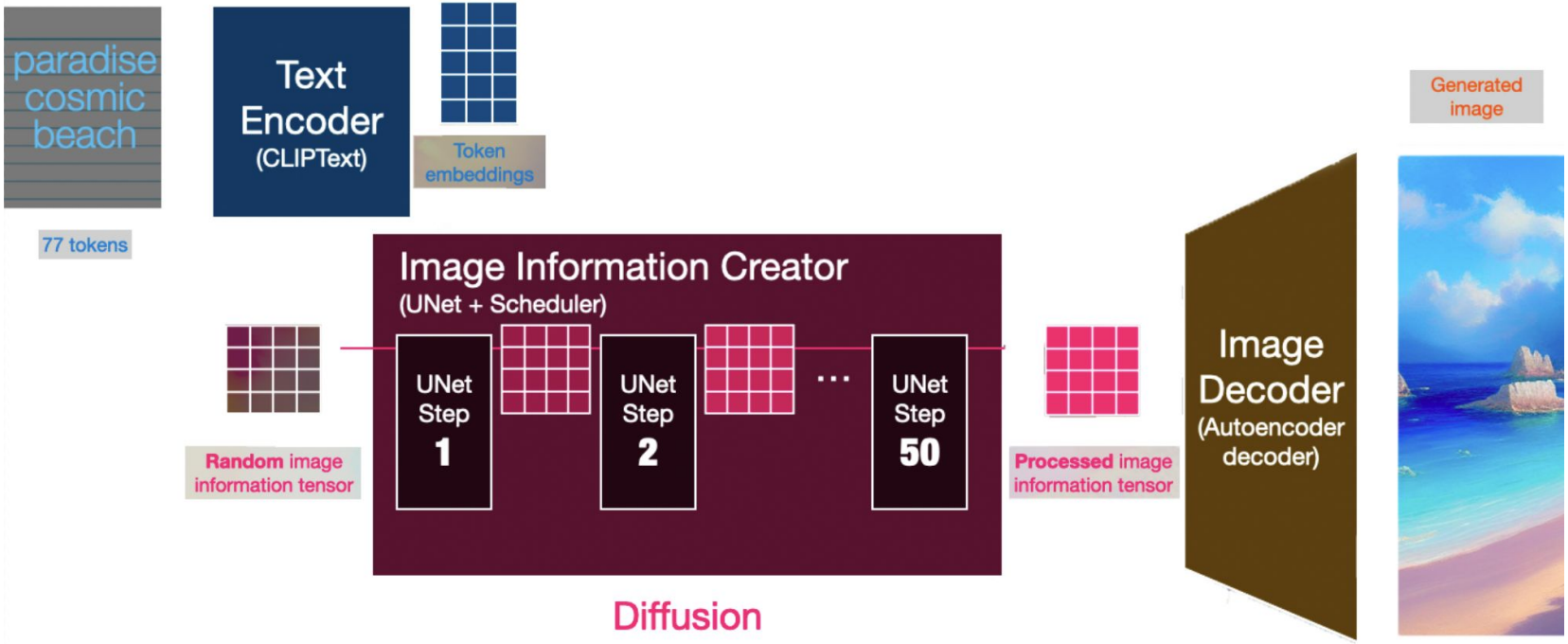
- механизм внимания в архитектуре Transformer, который смешивает две разные последовательности
- две последовательности должны иметь одинаковую размерность
- две последовательности могут иметь разную модальность (например, текст, изображение, звук)
- одна из последовательностей определяет длину вывода, так как она играет роль **query**
- другая последовательность является источником векторов **key value**



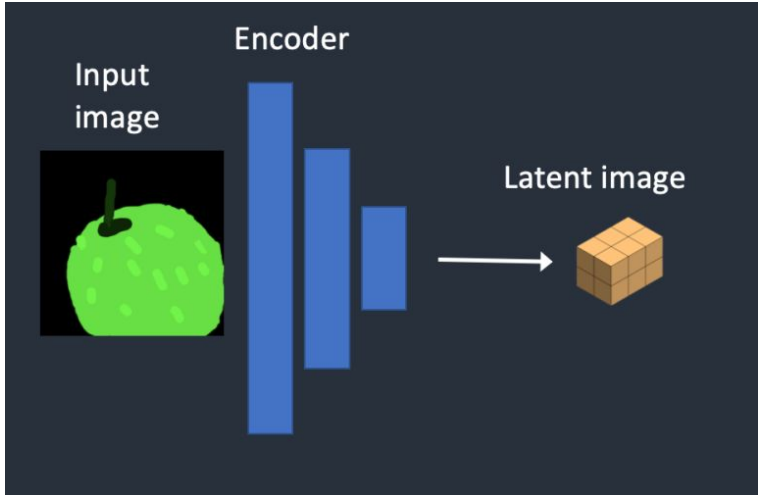
Cross-Attention in Transformer Decoder

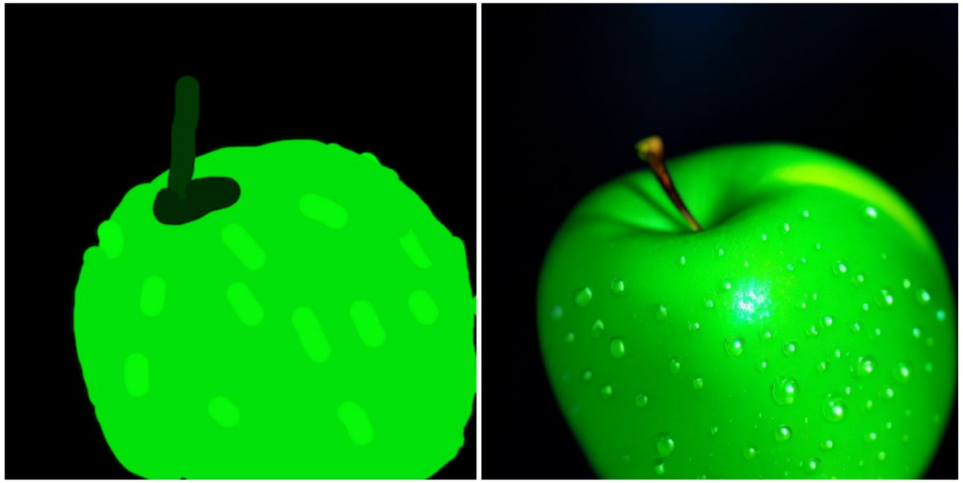


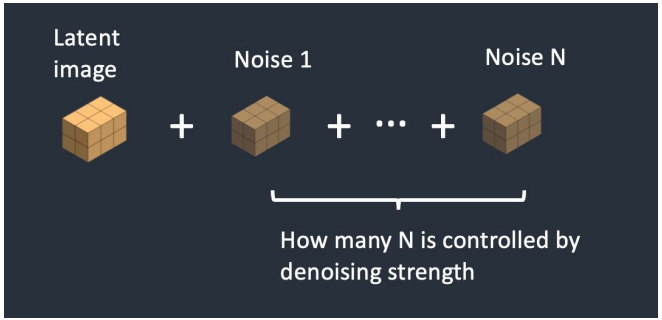
text2img


paradise
cosmic
beach
77 tokens
Text
Encoder
(CLIPText)
Token
embeddings
Image Information Creator
(UNet + Scheduler)
Random image
information tensor
UNet
Step
1
UNet
Step
2
...
UNet
Step
50
Processed image
information tensor
Image
Decoder
(Autoencoder
decoder)
Generated
image
Diffusion

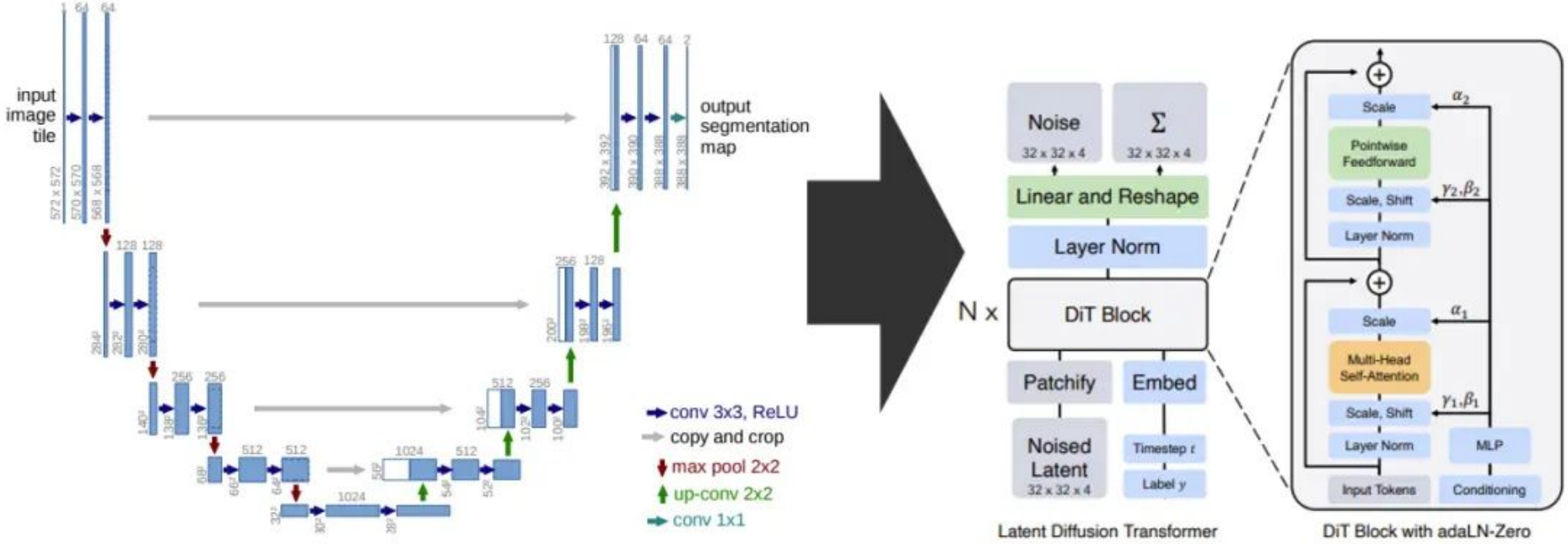
Img2img


Encoder
Input image
Latent image




Latent image
Noise 1
Noise N
+
+ ··· +
How many N is controlled by denoising strength

Core Idea of DiT



Stable Diffusion 1.4

- **Дата релиза:** август 2022
- **Архитектура:** Latent Diffusion Model (LDM)
- **Размер модели:** ~1.1 млрд параметров
- **Тренировочные данные:** LAION-2B (subset ~600M image-text pairs)
- **Особенности:**
    - Первая общедоступная версия.
    - Использует CLIP для текста и VAE для латентного пространства.
    - Текстовый ввод превращается в изображение с помощью U-Net.
    - Хорошо работала на обычных потребительских GPU (6-8 ГБ).

Stable Diffusion 1.5

- **Дата релиза:** октябрь 2022
- **Отличия от 1.4:**
  - Улучшенная фильтрация датасета.
  - Небольшие архитектурные и весовые доработки.
  - Стабильнее при генерации лиц и деталей.

Stable Diffusion 2.0 / 2.1

- **Дата релиза:** ноябрь-декабрь 2022
- **Ключевые изменения:**
  - **Новое разрешение:** обучение на 768x768 (vs 512x512 ранее).
  - **Новая текстовая модель:** OpenCLIP вместо CLIP.
  - **Новая VAE и U-Net архитектура.**
  - Появилась **depth-to-image** модель и **upscaler**.
- **Плюсы:**
  - Лучшая детализация и реализм.
- **Минусы:**
  - Некоторые промпты из 1.5 больше не работают («сломана» совместимость).
  - Меньшая "креативность" в абстрактных запросах.

SDXL (Stable Diffusion XL)

- **Дата релиза:** июль 2023
- **Размер:** 2 модели: Base (~3.5B) + Refiner (~6.6B)
- **Ключевые улучшения:**
  - Обработка изображений **1024x1024**.
  - Улучшена композиция, текст, анатомия.
  - Двухэтапный процесс: сначала Base генерирует изображение, затем Refiner улучшает детали.
  - Поддержка **negative prompts** улучшена.

Image Arena

Image arena
https://huggingface.co/spaces/ArtificialAnalysis/Text-to-Image-Leaderboard

Interpreting Stable Diffusion Using Cross Attention
https://nn.labml.ai/diffusion/stable_diffusion/sampler/ddpm.html